

Lab 03: Cluster analysis

Author: **N.J. de Winter** (*n.j.de.winter@vu.nl*)

Assitant Professor Vrije Universiteit Amsterdam

Statistics and Data Analysis Course

Learning goals:

- Apply and improve your knowledge of Python and Jupyter
- Get familiar with cluster analysis
 - Understand and apply tools to assess whether a dataset of *observations* can be clustered
 - Interpret *tree diagrams* based on datasets
- Develop a feeling for how statistical tools can help you, but you still require *your interpretation* to draw conclusions.

Introduction

In this lab assignment, you will learn to apply **cluster analysis** to a dataset. We will work with a dataset consisting of mineralogical analyses of sediments, a very common type of data for Earth Scientists! The tools you will start to work with in this lab are very useful for *classifying* datasets containing multiple (sometimes large amounts of) *observations* of multiple *variables*. The goal of **cluster analysis** is to group observations into *clusters*, not to combine *variables*. For combining *variables* we will work with **factor analysis** in the upcoming labs.

Since this is the third exercise, we will assume that you have a bit more experience with Python compared to last exercise. Don't forget that you can look up ways to load and adapt data you may need here in the previous labs in case you get lost.

As usual, we will start by loading some packages. Like in the previous labs, will need the `numpy` and `matplotlib` libraries again.

Exercise 1: Load the `numpy` package and the `pyplot` package (part of `matplotlib`) like you did in the previous labs. Don't forget that you also need to add the statement `%matplotlib inline` to allow plots to be visualized in Jupyter:

```
In [ ]: # Make sure our figures show up in Jupyter

# The 'numpy' package contains some handy functions
# The 'matplotlib' package contains tools needed to plot our data and results
```

Besides these common packages, we will also need to load the following:

```
In [ ]: import matplotlib.ticker as mt # A package in matplotlib that allows us to modify tick marks in plots
from scipy.spatial.distance import pdist, squareform # Some functions we need to calculate virtual "distances" between observations
from scipy.cluster import hierarchy as sch # A function we need to perform cluster analysis
```

Preparing your data

You have already learned how to define the working directory in previous labs. Do this if you are working in Spyder. If you are working in Jupyter (recommended), make sure the dataset `Lab03.txt` is in the same folder as your Jupyter Notebook.

The data we need for this lab is in `.txt` format, so we need a different command for loading it than the `.csv` data:

```
In [ ]: data = np.loadtxt('Lab03.txt', skiprows=1)
```

Make sure you explore your new dataset using the commands you have learned in previous Labs.

```
In [ ]: # Inspect your data
```

Question 1: How many observations (rows) does the dataset have? And how many measured parameters (columns)?

Answer 1:

The data is pretty bare, so it will be helpful to create labels for the observations and parameters. We will label the observations simply by calling them `Sample 1`, `Sample 2`, `Sample 3`, etc. You can create a *vector* of these names using the following commands:

```
In [ ]: # Create a vector of sample names
sample = ['Sample_' + str(i + 1) for i in range(10)]
```

Make sure you understand the code above. To help yourself, you can always use `print()` to look at the result.

```
In [ ]: print(sample)
```

The columns of the data sample represent the percentages of various minerals measured in the sediment samples. The sediments are sourced from 3 rock types:

1. a magmatic rock containing predominantly amphibole (`amp`), pyroxene (`pyr`), and plagioclase (`pla`)
2. a hydrothermal vein characterized by the occurrence of fluorite (`flu`), sphalerite (`sph`), and galenite (`gal`), as well as some feldspars (`pla`) and potassium feldspar (`ksp`) and quartz (`qtz`)
3. a sandstone unit containing `pla`, `ksp`, `qtz` and clay minerals (`cla`)

Your *parameters* in this dataset are the percentages of minerals measured in each sample. You can use the command below to create a vector of these mineral abbreviations in the order of the columns in your data:

```
In [ ]: # Create a vector of parameter names
minerals = ['amp', 'pyr', 'pla', 'ksp', 'qtz', 'cla', 'flu', 'sph', 'gal']
```

Inspecting the data structure

To test how your parameters (measurements of mineral content) correlate with each other, you can make a correlation matrix. You already looked at correlations between variables in datasets in `Lab01` and `Lab02` so you should know how to do this now using the `corr()` function (Look it up if you are not sure any more!). Since your data in this Lab originated from a `.txt` file and was not loaded using the `pandas` package, this syntax will not work. Instead you will need to use the more general `np.corrcoef()` function.

Exercise 2: Create a correlation matrix named 'corrmatrix' listing the correlations between the mineral content in your dataset and inspect the result.

```
In [ ]: # Create a correlation matrix of the mineral content
```

Question 2: Can you easily interpret this result to determine which minerals are correlated with each other?

Answer 2:

To make it easier to interpret the results, we will use the following string of commands to make a nice correlation plot. Make sure you follow exactly what is happening here. If you are unsure, you can always use the `help()` function or (usually more straightforwardly) just Google the functions to get information on what they do.

```
In [ ]: # First we flip the correlation matrix:
corrmatrix = np.flipud(corrmatrix)

# Second, we plot the correlation matrix with colors representing the degree of correlation:
plt.figure(1)
plt.imshow(corrmatrix, cmap = 'hot')

# Third, we add a title to the graph
plt.title('Correlation matrix')

# Fourth, we also add the mineral labels:
plt.xticks(np.arange(0, 9), [minerals[i] for i in range(9)])
plt.yticks(np.arange(0, 9), [np.flipud(minerals)[i] for i in range(9)])

# Finally, we display the colorbar as a legend:
plt.colorbar()
```

Question 3: Which (groups) of minerals are highly correlated? Does this correlation reflect the rock types in the dataset (see description above)

Answer 3:

Performing cluster analysis

To perform cluster analysis, we want to calculate the distances between pairs of samples. We will use the `pdist` function for that.

Exercise 3: Study the `pdist` function well using the help.

In []:

Question 4: Apparently, there are many different options to define the distance between pairs. Which parameter of the function `pdist()` allows you to set the parameter for defining the distance between observations? Which options for this parameter are familiar to you? (Hint: check the lecture slides) Can you define what these do?

Answer 4:

Let's first try the 'euclidean' distance.

Exercise 4: Following the syntax of the `pdist()` function you discovered using the `help()` function above (**Exercise 3**), calculate a vector `Y` of distances between all observations by applying the `pdist()` function on your dataset, defining the `metric` as `euclidean`. Inspect the result.

In []:

It would be easier to interpret this result if this was a distance matrix rather than a long vector of values, i.e. something similar to a correlation matrix. Luckily, there already exists a function to convert the distance vector to a matrix: `squareform`.

Exercise 5: Search the `help()` for `squareform` and apply the function on your new vector `Y` to create a distance matrix `X` of all the Euclidean distances. Use `print()` to inspect the result.

In []:

Exercise 6: Now we can plot the distance matrix `X` as a color image. Do so using the same steps as you followed to plot the correlation matrix above. Make sure you choose appropriate titles for the axes and the plot in general.

In []:

```
# Plot the distance matrix with colors representing the distance between samples:

# Add a title to the graph

# Add the sample labels:

# Display the colorbar as a legend:
```

In the plot above, dark/red colors denote pairs that are 'close' to each other, which means that these paired observations/samples are similar. Yellow-to-white observation show pairs that are 'far' from each other, which means that these samples are quite different.

Hint: Of course you can use different color scales if you want, just check the `help()` of the function `plt.imshow` and look for the options you have for the parameter `cmap`

In []:

```
help(plt.imshow)
```

We now want to construct a tree diagram, and we therefore need a hierarchical algorithm to cluster observations in an iterative manner. Linkage is a function that does this.

Exercise 7: Search the help for the `sch.linkage` function and check which parameters of the function you can play around with (scroll down to the parameters section).

In []:

We can define the linkage function with several input parameters, such as the `method` and `metric`. The method defines the algorithm for computing distances between clusters. Some of these options are easy to understand, like `average`, `complete`, `single`. Others may need some more thinking. With the `metric` option we can again define a distance metric such as `euclidean`, `cityblock` or `correlation`.

Exercise 8: Apply the `sch.linkage` function using the `method` 'single' and the `metric` 'euclidean' on the dataset and assign the result to an object called `Z`. Then inspect the resulting object `Z`.

In []:

Understanding the output `Z` requires some attention: The first row denotes the first cluster that was formed. The numbers in the first 2 columns of row 1 show which initial clusters (now still individual samples) that were clustered. Since indices in Python start with 0, note that index 0 refers to sample 1, index 1 to sample 2, etc. Thus, the first cluster joined samples 2 and 9. In the third column of row 1 we can see their paired distance. The first row thus shows the first step the clustering algorithm took.

For further clustering: this newly derived cluster needs a new label. This new label is simply the number of initial samples (10) + the clustering step/row (here 1). The newly formed cluster thus gets assigned the label 11. Now we can continue with clustering. The second cluster joined samples 8 and 10, the resulting new cluster is labeled 12 (10 original samples + 2 steps). The third cluster joined sample 1 with cluster 12. The newly formed cluster is labeled 13. Etcetera.

We can visualize this clustering tree using the dendrogram function. The code below looks complicated, so make sure you read it line by line and use the comments (denoted by the '#') to understand what's going on. You can always copy parts of the code in a new code cell and/or inspect the results to test your understanding.

```
In [ ]: plt.figure(2, figsize=(20, 15)) # Create a new plot with size x = 20, y = 15)
ax = plt.gca() # "gca" means "get current axes", so this saves the axes of the plot in the object "ax"
dn = sch.dendrogram(Z, labels = sample, ax = ax) # Create a dendrogram using the Linkage data in object "Z" you just created and Labeling the samples using your "sample" vector
ax.set_xlabel('Sample number', {'fontname':'Calibri', 'fontsize':14}) # Label the x-axis + Layout
ax.set_ylabel('Single Euclidean distance', {'fontname':'Calibri', 'fontsize':14}) # Label the y-axis + Layout
ax.xaxis.set_major_locator(mtl.FixedLocator(np.arange(5, 10 * 10 + 5, 10))) # Set the Locations of tick marks on the x-axis
ax.yaxis.set_major_locator(mtl.MultipleLocator(base = 0.05)) # Set the Locations of tick marks on the y-axis
ax.yaxis.set_minor_locator(mtl.AutoMinorLocator()) # Set the Locations of minor tick marks on the y-axis
ax.tick_params(axis = 'y', which = 'both', direction = 'in', left = True, right = True) # Set layout parameters (title, direction, alignment, etc) of x-axis
ax.tick_params(axis = 'x', which = 'major', direction = 'in', top = True, bottom = True) # Set layout parameters (title, direction, alignment, etc) of y-axis
plt.title('Dendrogram of 10 sediment samples', {'fontname':'Calibri', 'fontsize':24}) # Create a plot title + Layout
```

This graph visualizes everything from matrix `Z`.

Question 5: Does the dendrogram yield clear groups of samples? If so, which samples cluster together? Can you make sense of these groups based on what you know about the samples in your dataset?

Answer 5:

Question 6: Where (i.e. at which euclidean distance value) would you cut off the tree to withhold clusters?

Answer 6:

A result like this might be dependent on the choices you made while doing the clustering. Doing a rigorous statistical analysis requires testing of the effect of such choices on the analysis.

Question 7: Can you think of a choice you made when making the dendrogram above that could affect your result?

Answer 7:

BONUS exercise: Repeat the cluster analysis above using the distance metrics Manhattan distance (`cityblock`) and Pearson's correlation (`correlation`) and plot the results to check whether the clustering you ended up with is robust against changes in the metric.

```
In [ ]: # Create a new linkage file using the Manhattan distance

# Inspect the result

# Plot the dendrogram using Manhattan distance
# Create a new plot with size x = 20, y = 15)
# "gca" means "get current axes", so this saves the axes of the plot in the object "ax"
# Create a dendrogram using the Linkage data in object "Z" you just created and Labeling the samples using your "sample" vector
# Label the x-axis + Layout
# Label the y-axis + Layout
# Set the Locations of tick marks on the x-axis
# Set the Locations of tick marks on the y-axis
# Set the Locations of minor tick marks on the y-axis
# Set layout parameters (title, direction, alignment, etc) of x-axis
# Set layout parameters (title, direction, alignment, etc) of y-axis
# Create a plot title + Layout
```

```
In [ ]: # Create a new linkage file using the Pearson's correlation as distance

# Inspect the result

# Plot the dendrogram using Manhattan distance
# Create a new plot with size x = 20, y = 15)
# "gca" means "get current axes", so this saves the axes of the plot in the object "ax"
```

```
# Create a dendrogram using the Linkage data in object "Z" you just created and labeling the samples using your "sample" vector
# Label the x-axis + layout
# Label the y-axis + layout
# Set the locations of tick marks on the x-axis
# Set the locations of tick marks on the y-axis
# Set the locations of minor tick marks on the y-axis
# Set layout parameters (title, direction, alignment, etc) of x-axis
# Set layout parameters (title, direction, alignment, etc) of y-axis
# Create a plot title + layout
```

Conclusion: